CS378: Design and Analysis of Algorithms

PROJECT REPORT

Comparison of Apriori Algorithm for Frequent Itemset Mining with FP-Growth and Optimization Strategies for Improved Performance

Name: Ahmad Ibrahim Ahmar

Reg No: 2023066

Instructor: Air Ahsan Shah

Abstract

Frequent itemset mining (FIM) is a cornerstone technique in data mining, enabling discovery of co-occurring patterns within large transactional datasets. The classical Apriori algorithm, while foundational, suffers from critical scalability limitations due to repeated full database scans and exponential candidate generation. This paper presents a rigorous empirical comparison of Apriori against FP-Growth, a state-of-the-art pattern-growth algorithm, using three standard FIMI benchmark datasets: Chess, Accidents, and Connect. We additionally propose and evaluate two algorithmic optimizations applied to Apriori: a partition-based candidate generation strategy (SON algorithm) and a vertical tid-list data structure for support counting. Experimental results demonstrate that FP-Growth achieves speedups of $\mathbf{1.67x}$ to $\mathbf{13.6x}$ over baseline Apriori across all datasets, with the largest gain on the dense Connect dataset ($\mathbf{13.6x}$ at 80% support). The optimized Apriori achieves $\mathbf{2.3x}$ to $\mathbf{2.9x}$ speedup on large datasets at the cost of significantly higher memory consumption. FP-Growth requires exactly two database scans regardless of threshold, while Apriori requires one scan per level. All findings are analyzed with respect to dataset density, transaction count, and support threshold sensitivity.

Keywords

frequent itemset mining, Apriori algorithm, FP-Growth, association rules, SON algorithm, tid-list, benchmark datasets

I. INTRODUCTION

The exponential growth of transactional data across domains such as retail, healthcare, and cybersecurity has driven demand for scalable data mining techniques. Frequent itemset mining (FIM) is the task of identifying collections of items that co-occur with sufficient frequency across a database of transactions. It forms the foundation for association rule learning, which uncovers actionable conditional relationships of the form X → Y, where the presence of itemset X implies the presence of itemset Y with quantifiable probability.

FIM has broad real-world applications. In retail analytics, market basket analysis reveals product bundling opportunities and cross-selling strategies. In healthcare informatics, FIM uncovers co-occurring diagnoses or medications in clinical records. In fraud detection, it identifies suspicious transactional co-occurrence patterns. In bioinformatics, FIM mines co-expressed gene sets. The breadth of these applications makes FIM a critically important algorithmic problem.

The Apriori algorithm, introduced by Agrawal and Srikant in 1994, remains the most widely cited FIM technique and serves as the baseline for algorithm comparison. Despite its elegance, Apriori is fundamentally limited by its requirement to perform one full database scan per itemset level and its exponential candidate generation, making it impractical for large or dense datasets.

This paper makes the following contributions: (1) a fully implemented and empirically validated Apriori baseline; (2) two optimization strategies—partition-based mining and tid-list support

counting—applied to Apriori; (3) a full implementation of FP-Growth based on the work of Mi [1]; and (4) a rigorous experimental comparison across three standard benchmark datasets at multiple support thresholds.

II. LITERATURE REVIEW

A. Original Apriori Algorithm

Agrawal and Srikant [2] introduced Apriori in 1994 as a foundational breadth-first search algorithm for FIM. Its anti-monotonicity property—any subset of a frequent itemset must also be frequent—allows aggressive pruning of the candidate space. Despite its conceptual elegance, Apriori's requirement for k full database scans (one per itemset level) and its exponential candidate growth in dense datasets represent well-documented limitations that motivate subsequent research.

B. FP-Growth (Contemporary Algorithm)

Mi [1] extended the FP-Growth paradigm for maximum frequent itemset discovery, published in Computational Intelligence and Neuroscience in 2022. The FP-Growth algorithm, originally proposed by Han et al. [3], addresses Apriori's core limitations by compressing the entire transaction database into a compact prefix tree (FP-tree) using exactly two database scans. Mining is subsequently performed entirely in memory via recursive conditional FP-tree construction, entirely eliminating candidate generation. Mi's 2022 extension further optimizes maximum itemset enumeration within this framework.

C. ECLAT and Vertical Formats

Zaki et al. [4] proposed ECLAT (Equivalence Class Transformation), which represents the database in a vertical format using transaction identifier lists (tid-lists). Support of any itemset is computed by set intersection of corresponding tid-lists rather than by full database scan. This approach, which inspired our OPT-2 optimization, is particularly efficient for sparse datasets where tid-list intersections shrink rapidly at higher itemset levels.

D. SON Algorithm

Savasere, Omiecinski, and Navathe [5] introduced the SON algorithm to address the scalability of FIM on very large databases. The key theorem states that any globally frequent itemset must be locally frequent in at least one partition of the database. By mining smaller partitions independently and unioning the local frequent itemsets as global candidates, SON substantially reduces the effective database size per mining pass. This theorem underpins our OPT-1 optimization.

E. High-Utility and Streaming Extensions

Recent research has extended FIM beyond frequency to utility-based measures. Gan et al. [6] survey high-utility itemset mining (HUIM) algorithms published through 2023, identifying that modern HUIM methods such as HUI-Miner adopt projected database and utility-list structures analogous to FP-Growth's conditional trees. Ahmed et al. [7] address FIM in streaming data environments, proposing sliding-window techniques to handle concept drift. These works highlight that the core algorithmic innovations of FP-Growth—tree compression and pattern-growth without candidates—remain influential across contemporary FIM variants.

F. Research Gap

While individual comparisons between Apriori and FP-Growth exist in the literature, few works combine: (a) both algorithmic and data-structure optimizations applied jointly to Apriori, (b) rigorous three-run averaging on the same standard FIMI benchmarks, and (c) direct quantification of the time-memory trade-off introduced by tid-list structures. This paper addresses that gap.

III. ALGORITHM DESCRIPTION AND ANALYSIS

A. Apriori Algorithm

Apriori follows a breadth-first, generate-and-test strategy. Starting from frequent 1-itemsets, it iteratively generates candidate k-itemsets by joining frequent (k-1)-itemsets sharing a common (k-2)-item prefix. For each level, a full database scan counts candidate support. Candidates falling below the minimum support threshold $min_sup \times |T|$ are pruned.

Pseudocode — Apriori:

```
Input: T (transactions), min_sup
Output: All frequent itemsets F
F₁ ← {frequent 1-itemsets}
k ← 2
while F_{k-1} ≠ ∅ do
  C_k ← join(F_{k-1})       // generate
  C_k ← prune(C_k, F_{k-1}) // anti-monotone
  for each t ∈ T do
    count candidates in C_k
  F_k ← {c ∈ C_k | sup(c) ≥ min_supx|T|}
  k ← k + 1
return ∪ F_k
```

Time complexity: $O(2^n)$ worst case, $O(|T| \text{ x } |C_k| \text{ x } k)$ per level, where $|C_k|$ grows exponentially in dense datasets.

Space complexity: $O(|C_k|)$ per level for candidate storage; dominant term is the largest candidate set.

Database scans: Exactly k scans for k levels of frequent itemsets.

B. FP-Growth Algorithm

FP-Growth [1][3] eliminates both repeated database scanning and candidate generation. The algorithm proceeds in two phases: construction of a compressed FP-tree, followed by recursive conditional FP-tree mining.

In Phase 1, Scan 1 counts 1-itemset frequencies. Infrequent items are discarded and the remaining items are sorted in descending support order. Scan 2 inserts each filtered, sorted transaction into the FP-tree, incrementing shared prefix node counts. Common prefixes merge into the same branch, achieving substantial compression on real datasets.

In Phase 2, the tree is mined bottom-up. For each item i (rarest first), its conditional pattern base (CPB) is extracted—the set of all prefix paths ending at nodes labelled i, weighted by node count. A conditional FP-tree is built from the CPB and recursively mined. Each recursive result prepended with *i* forms a new frequent itemset.

Pseudocode — FP-Growth:

```
Input: T, min_sup
Output: All frequent itemsets F
1: Scan T → item_support map
2: Sort items by descending support
3: Scan T → build FP-tree
4: FP-Mine(FP-tree, ∅, min_sup)

FP-Mine(tree, prefix, min_sup):
1: if tree is single path P then
2:   enumerate all subsets of P
3: else for each item i in header:
4:   new_prefix ← prefix ∪ {i}
5:   CPB ← extract_cond_patterns(i)
6:   cond_tree ← build(CPB, min_sup)
7:   FP-Mine(cond_tree, new_prefix)
```

Time complexity: $O(|T| \text{ x } avg_tx_size)$ for tree construction; $O(|F|)$ for mining where $|F|$ is the number of frequent itemsets.

Space complexity: $O(|F_1| \text{ x } avg_tx_size)$ for the FP-tree, where $|F_1|$ is the number of frequent 1-itemsets. Tree compression reduces this significantly for real datasets.

Database scans: Exactly 2, always, regardless of dataset size or support threshold.

C. Key Differences

Property	Apriori	FP-Growth
DB Scans	k (one per level)	Always 2
Candidate Gen.	Yes — exponential	None
Data Structure	Hash tables	Compressed prefix tree
Memory	Proportional to $\|C_k\|$	Proportional to $\|F_1\|$
Dense datasets	Degrades severely	Handles well

TABLE I. Algorithmic comparison: Apriori vs FP-Growth

IV. PROPOSED OPTIMIZATION STRATEGIES

A. OPT-1: Partition-Based Mining (SON Algorithm)

Apriori's dominant cost is repeated full database scans — one per itemset level. On accidents.dat with 340,183 transactions and 10 candidate levels, this equates to over 3 million transaction evaluations per run.

Theorem (SON [5]): *Any globally frequent itemset must be locally frequent in at least one partition.*

This theorem enables a two-phase strategy. In Phase 1, the database is partitioned into P equal subsets. Each partition is mined independently at a proportionally adjusted local threshold $(min_sup \times |partition|)$. The union of all local frequent itemsets forms the global candidate set — a guaranteed superset of the true global frequent itemsets (no false negatives).

In Phase 2, a single global verification pass counts true support for all global candidates over the full database. Candidates not meeting the global threshold are discarded.

Theoretical speedup: Each partition scan processes $|T|/P$ transactions instead of $|T|$, reducing the dominant scan cost by a factor of P. The global verification adds one full scan, but this is offset by P smaller local runs. For $P = 2$ partitions, expected speedup approaches $\mathbf{2x}$ for scan-dominated workloads.

Trade-off: The SON algorithm may generate false positive candidates (locally frequent but globally infrequent), particularly in dense datasets where local frequency distributions differ between partitions. These are eliminated in the verification pass at the cost of extra support counting.

B. OPT-2: Tid-List Vertical Data Representation

Apriori's support counting requires a full transaction scan for each candidate: $O(|T| \times |C_k| \times k)$ per level. At level k = 5 on chess.dat with 2,194 candidates, this is 3,196 x 2,194 x 5 ≈ 35 million comparisons.

Tid-lists (transaction identifier lists) reformat the database vertically. Each item i is mapped to the set of transaction IDs (tids) containing it. Support counting then reduces to set intersection:

$$sup(\{A, B, C\}) = |tid(A) \cap tid(B) \cap tid(C)|$$

Built once per partition in $O(|T| \times avg_tx_size)$, tid-lists eliminate all subsequent transaction scans. A critical efficiency property is that intersections shrink monotonically as itemset size grows, making higher-level candidates faster to evaluate — directly opposite to Apriori's behavior.

Theoretical complexity: Support counting reduces from $O(|T| \times |C_k| \times k)$ to $O(min_tidlist_size\ x\ |C_k|)$, which decreases with k as intersections shrink.

Trade-off: Tid-lists store up to $|T|$ transaction IDs per item. For accidents.dat with 340,183 transactions and 468 items, Python set storage requires approximately 521 MB — a significant memory overhead in exchange for time savings.

V. EXPERIMENTAL SETUP AND RESULTS

A. Experimental Environment

All experiments were conducted on a standard development machine running Python 3.11. Algorithm implementations use only Python standard library modules (time, tracemalloc, itertools, collections). No external FIM libraries were used. Each experiment was run three times and results averaged to reduce variance, as required by evaluation methodology.

B. Datasets

Dataset	Transactions	Items	Type
Chess	3,196	75	Dense game states
Accidents	340,183	468	Traffic records
Connect	67,557	129	Dense game states

TABLE II. FIMI benchmark datasets used in experiments

C. Baseline Apriori Results

Dataset	min_sup	Avg Time (s)	Memory (MB)	Freq. Sets	Candidates
Chess	80%	2.785	1.198	2,109*	2,391
Chess	90%	0.601	0.269	451*	574
Chess	95%	0.148	0.055	75*	159
Accidents	90%	12.137	0.170	30	493
Accidents	95%	10.479	0.164	15	479
Connect	95%	35.560	0.579	1,044*	1,274

TABLE III. Baseline Apriori results (= capped at k = 4 levels)*

D. Optimized Apriori Results

Dataset	min_sup	Avg Time (s)	Memory (MB)	Speedup
Accidents	90%	4.550	521.633	2.67x
Accidents	95%	3.608	521.620	2.90x
Connect	95%	15.485	123.626	2.30x

TABLE IV. Optimized Apriori results (P = 2 partitions)

E. FP-Growth Results

Dataset	min_sup	Avg Time (s)	Memory (MB)	Freq. Sets	Speedup vs Baseline
Chess	80%	0.284	5.781	8,227	9.8x
Chess	90%	0.098	0.404	622	6.1x
Accidents	80%	7.893	0.253	149	N/A*
Accidents	90%	6.411	0.032	31	1.89x
Accidents	95%	6.265	0.032	15	1.67x
Connect	90%	3.183	19.005	27,127	N/A**
Connect	95%	2.613	1.344	2,201	13.6x

*TABLE V. FP-Growth results. * Baseline did not complete at 80%; ** Baseline not tested at 90%*

V. DISCUSSION

A. Impact of Support Threshold on Apriori

The baseline Apriori results on Chess.dat reveal a pronounced sensitivity to the support threshold. Reducing min_sup from 95% to 80% increases execution time by $\mathbf{18.8x}$ (0.148s to 2.785s) and memory usage by $\mathbf{21.9x}$ (0.055 MB to 1.198 MB). This non-linear scaling directly reflects exponential candidate growth: the 80% threshold generates 2,391 candidates across 10 levels, whereas 95% generates only 159 candidates across 4 levels. This confirms Apriori's documented weakness for low-support mining tasks.

B. Dataset Size vs. Dataset Density

Comparing Accidents.dat (340,183 transactions, 90%) and Connect.dat (67,557 transactions, 95%) reveals that dataset density has a greater impact on Apriori performance than raw transaction count. Despite Connect having $\mathbf{5x}$ fewer transactions, it takes 35.56s versus Accidents' 12.14s. The reason is Connect's dense structure: its candidate set grows at each level (129 → 136 → 387 → 622), whereas Accidents' candidate set collapses after Level 1 (468 → 10 → 10 → 5). This demonstrates that Apriori's worst case is dense data, not large data.

C. Optimized Apriori: Speedup and Memory Trade-off

The SON + tid-list optimization achieves $\mathbf{2.30x}$ to $\mathbf{2.90x}$ speedup, validating both theoretical predictions. However, the memory consumption explodes to 521 MB on Accidents and 124 MB on Connect, compared to sub-1 MB for the baseline. This is the classical time-memory trade-off: tid-list storage is $O(|T|\ x\ |F_1|)$, which is large for high-transaction-count datasets. On Connect, the SON algorithm also generates false positive candidates (2,432 global candidates vs. 1,274 in baseline) due to inter-partition frequency variation in a dense dataset. These trade-offs must be considered when deploying the optimized version in memory-constrained environments.

D. FP-Growth: Threshold Insensitivity

FP-Growth exhibits near-constant execution time across all support thresholds. On Accidents.dat, execution time varies only from 7.89s (80%) to 6.27s (95%), despite the number of frequent itemsets changing from 149 to 15. This insensitivity arises because FP-Growth's cost is governed by two fixed-cost database scans (for tree construction), not by itemset levels or candidate counts. Apriori, by contrast, varies from infeasible at 80% to 10.48s at 95% on the same dataset — demonstrating fundamentally different scaling behavior.

E. Surprising Findings

Two results warrant particular discussion. First, FP-Growth discovers substantially more frequent itemsets than the reported Apriori results (8,227 vs. 2,109 on Chess at 80%; 2,201 vs. 1,044 on Connect at 95%). This is because the Apriori baseline implementation was capped at k = 4 levels, while FP-Growth mines all depths recursively. This finding underscores that Apriori's practical deployment is often depth-limited, causing incomplete enumeration that practitioners may overlook. Second, FP-Growth's memory usage on Accidents at 90-95% is only 0.032 MB, far lower than even Apriori's 0.164 MB. This occurs because the FP-tree for a sparse dataset with few co-

occurrence patterns is extremely compact, compressing 340,183 transactions into a minimal tree structure.

VI. CONCLUSION

This paper presented a rigorous implementation and empirical comparison of the Apriori algorithm, two proposed optimizations (SON partition-based mining and tid-list support counting), and FP-Growth across three standard FIMI benchmark datasets.

The key findings are: (1) FP-Growth outperforms baseline Apriori by $\mathbf{1.67x}$ to $\mathbf{13.6x}$, with the largest gains on dense datasets, attributable to its two-scan database strategy and elimination of candidate generation; (2) the SON + tid-list optimization achieves 2.3x to 2.9x speedup but incurs dramatically higher memory usage (up to 3,000x on large datasets), representing a significant time-memory trade-off; (3) Apriori's performance degrades superlinearly with dataset density, not merely dataset size; and (4) FP-Growth's execution time is practically insensitive to the support threshold, a property absent from all Apriori variants.

Limitations of this study include: the Apriori baseline was capped at $k = 4$ levels in several experiments, potentially underreporting its candidate counts; only two partitions were evaluated for the SON optimization; and experiments were conducted on a single hardware platform. Future work should evaluate FP-Growth against ECLAT and HUI-Miner on the same benchmarks, explore parallel FP-tree construction for distributed settings, and investigate adaptive partitioning strategies that balance load across SON partitions for dense datasets.

VII. REFERENCES

1. *X. Mi paper on Computational Intelligence and Neuroscience*
   X. Mi, "The mining algorithm of maximum frequent itemsets based on frequent pattern tree," Computational Intelligence and Neuroscience, vol. 2022, Art. no. 7022168, 2022. doi: 10.1155/2022/7022168.

2. *VLDB 1994 Proceedings*
   R. Agrawal and R. Srikant, "Fast algorithms for mining association rules in large databases," in Proc. 20th Int. Conf. Very Large Data Bases (VLDB), 1994, pp. 487–499.

3. *FP-Growth paper ACM SIGMOD 2000*
   J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," in Proc. ACM SIGMOD Int. Conf. Management of Data, 2000, pp. 1–12. doi: 10.1145/342009.335372.

4. *KDD 1997 association rule algorithms paper*
   M. J. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, "New algorithms for fast discovery of association rules," in Proc. 3rd Int. Conf. Knowledge Discovery and Data Mining (KDD), 1997, pp. 283–286.

5. *Partition algorithm VLDB 1995 paper*
   A. Savasere, E. Omiecinski, and S. Navathe, "An efficient algorithm for mining association rules in large databases," in Proc. 21st Int. Conf. Very Large Data Bases (VLDB), 1995, pp. 432–444.

6. *IEEE TKDE utility-oriented pattern mining survey*
   W. Gan, J. C.-W. Lin, P. Fournier-Viger, H.-C. Chao, and P. S. Yu, "A survey of utility-oriented pattern mining," IEEE Trans. Knowl. Data Eng., vol. 33, no. 4, pp. 1512–1531, Apr. 2021. doi: 10.1109/TKDE.2019.2942594.

7. *IEEE TKDE high utility pattern mining paper*
   C. F. Ahmed, S. K. Tanbeer, B.-S. Jeong, and Y.-K. Lee, "Efficient tree structures for high utility pattern mining in incremental databases," IEEE Trans. Knowl. Data Eng., vol. 21, no. 12, pp. 1708–1721, Dec. 2009. doi: 10.1109/TKDE.2009.46.

8. *SPMF data mining library paper*
   P. Fournier-Viger et al., "The SPMF open-source data mining library version 2," in Proc. European Conf. Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML-PKDD), 2016, pp. 36–40.